

## BFS and DFS :

### ① BFS (Breadth First Search) :

graph/tree traversal algo explores nodes level by level.

### ② working (uses a queue FIFO) :

- i. start from source node
- ii. mark visited
- iii. Pushed into queue
- iv. Remove front node
- v. visit all unvisited neighbors and push to queue.
- vi. repeat until queue becomes empty.

### ③ Queue, visited array/set.

(level order traversal)

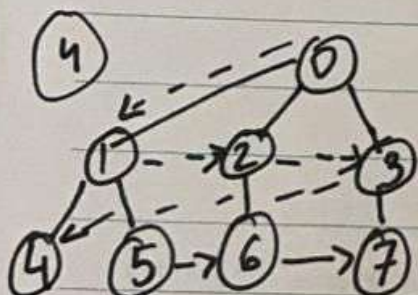
$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$

time complex :  $O(b^d)$

space " :  $O(b^d)$

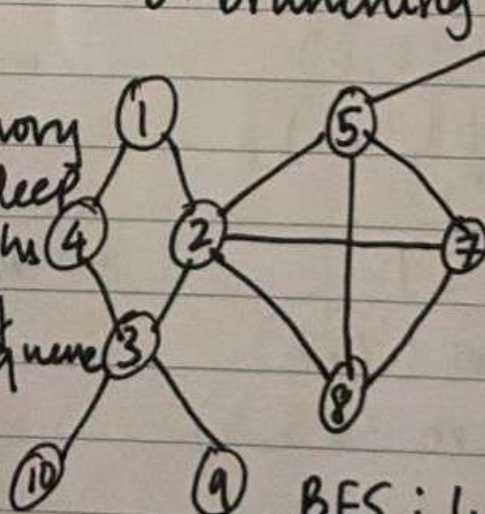
b : branching factor

d : depth of soln



### ④ disadv.:

- ⑤ high memory
- slow for deep graphs
- stores many nodes in queue



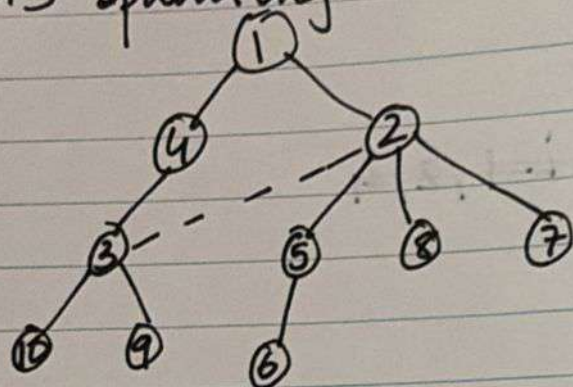
BFS : 1, 4, 2, 3, 5, 8, 7, 10, 9, 6

Queue :

|   |   |   |   |   |   |   |    |   |   |
|---|---|---|---|---|---|---|----|---|---|
| 1 | 4 | 2 | 3 | 5 | 8 | 7 | 10 | 9 | 6 |
|---|---|---|---|---|---|---|----|---|---|



## ⑥ BFS Spanning Tree :



## ⑦ adv.:

- shortest path
- simple to imp.
- good for order level trav.
- complete algo

## ⑧ applications : shortest path, social network, GPS/navigation

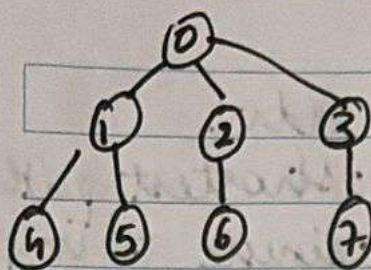
## ⑨ types :

- ① standard - normal, graph & tree traversal
  - ② multi source - multiple start nodes, rotten oranges problem, distance from nearest node
  - ③ bidirectional - source and distance runs from stops when both meet.  $O(b^{d/2})$ . faster, shortest path problem
  - ④ level order - special for trees, Binary Trees  
level 1  $\rightarrow$  level 2  $\rightarrow$  level 3.
- ## ⑩ Uninformed Search Algorithm (non heuristic)

## Depth First Search (DFS) :

- ① explores as deep as possible before backtracking.
- ② Working (Stack or Recursion (imp stack)) :
  - i. start from source node.
  - ii. visit node.
  - iii. move to unvisited neighbor.
  - iv. continue deeply.
  - v. If dead end reached  $\rightarrow$  backtrack.



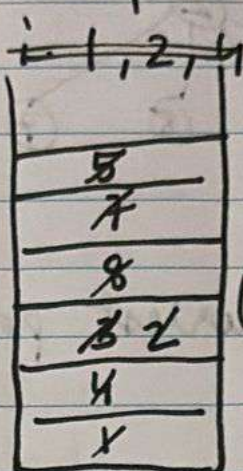
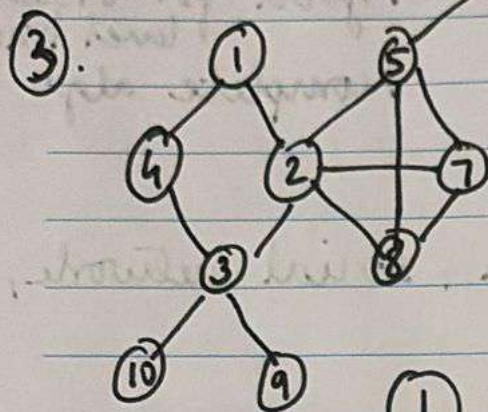


0, 1, 4, 5, 2, 6, 3, 7

time comp:  $O(b^d)$

space comp:  $O(bm)$

m: max depth



DFS: 1, 4, 3, 10, 9, 2, 8, 7, 5,

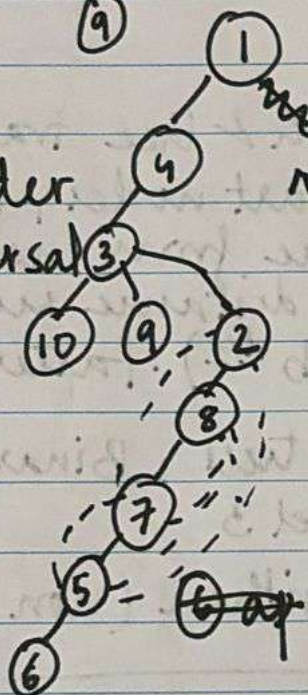
④ adv:

- less memory
- good for deep soln
- useful for backtrack
- recursive easy imp.

⑤ disadv.

- Spanning Tree • does not guarantee shortest path
- can get stuck deep.
- recursive stack overflow.

preorder traversal



back edges

⑥ applications: cycle detect, topological sort, maze solving, path finding, deadlock detection, backtracking, connected components.

⑦ Types:

- ① Recursive DFS: recursion/call stack, simpler implementation.
- ② Iterative DFS: explicit stack, avoids overflow, no recursion.



- ③ depth limited search: DFS with fixed depth limited, prevents infinite traversal, but may miss solution beyond limit but won't get stuck in the during traversal.
- ④ Iterative Deepening DFS [IDDFS]: repeated dfs increasing depth limit, combined BFS completeness and DFS low memory.
- ⑤ Backtracking DFS: wrong paths are backtracked.  
n queens, sudoku, maze solving.

---

DFS - for deep soln, can go inf. deep, not opt.

BFS - for shortest path, memory consumption, opt.

---



- ③ depth limited search: DFS with fixed depth limited, prevents infinite traversal, but may miss solution beyond limit but won't get stuck in the during traversal.
- ④ Iterative Deepening DFS [IDDFS]: repeated dfs increasing depth limit, combines BFS completeness and DFS low memory.
- ⑤ Backtracking DFS: wrong paths are backtracked. n queens, sudoku, maze solving.

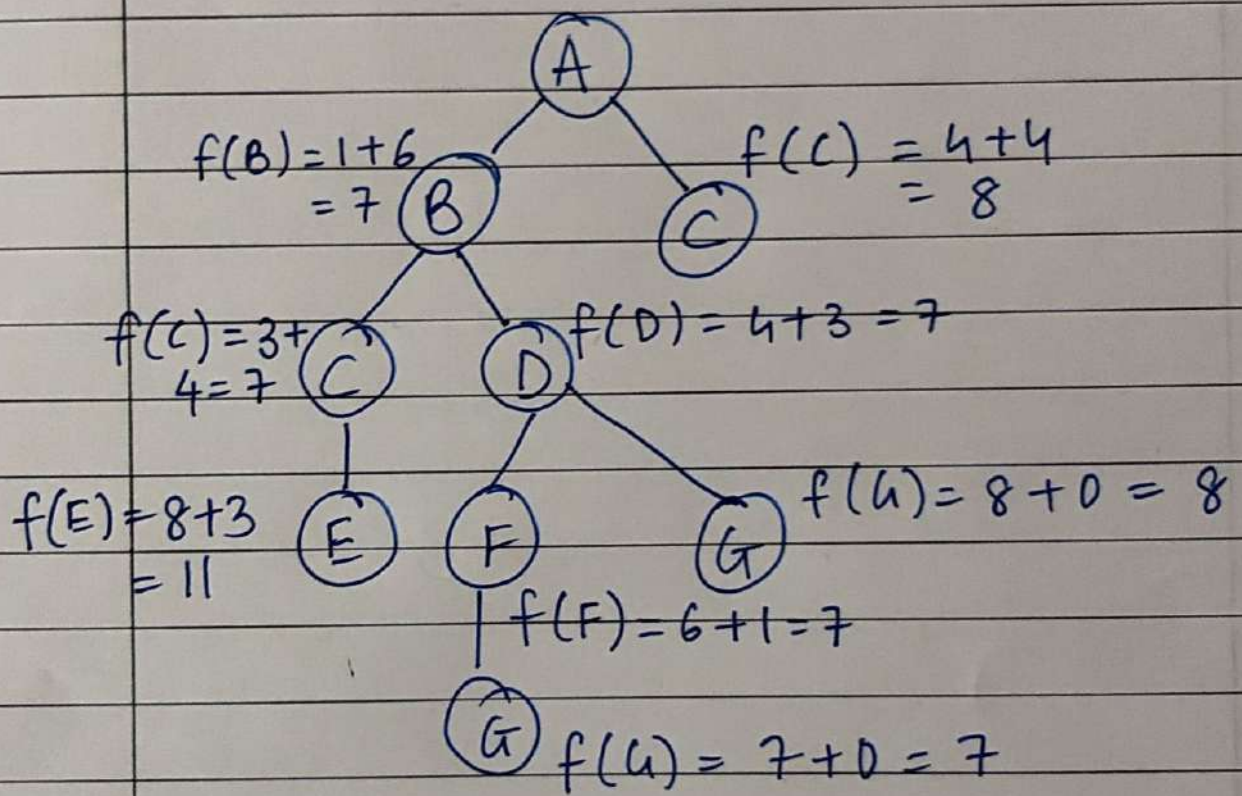
BFS - for deep soln, can go inf. deep, not opt.

BFS - for shortest path, memory consumption, opt

- ① A\* Algorithm: informed search algo, to find shortest optimal path between start node and given node.
  - uses actual cost travelled + estimated cost to goal to intelligently choose paths.
- ② Main Idea: chooses node with minimum,
 
$$f(n) = g(n) + h(n) \rightarrow \text{heuristic est from cur node to goal.}$$

total est cost; actual cost from start to cur node
- ③ Intuition: google maps Pune  $\rightarrow$  Mumbai  
A\* considers dist already travelled + est remaining distance and then chooses promising road
- ④ Components:
  - i.  $g(n)$ : actual cost from start to end  
example:  $S \rightarrow A = 4$ ,  $S \rightarrow A \rightarrow B = 7$  then  $g(B) = 7$
  - ii.  $h(n)$ : heuristic est from cur to goal, approx.  
examples: straightline dist, manhattan dist, euclidean distance.
  - iii.  $f(n)$ : total estimate, A\* always expands node with smallest  $f(n)$ .







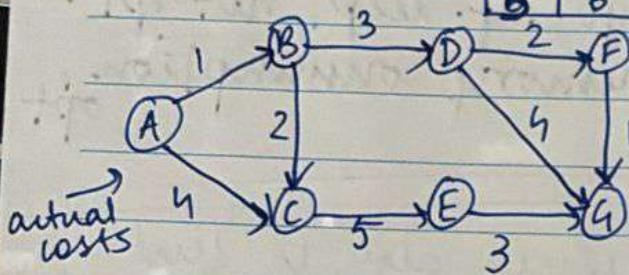
- ⑤ Working (uses OPEN list and ~~closed list~~ CLOSED list)
- i. put start node to OPEN list
  - ii. calculate  $f(n)$
  - iii. pick node with smallest  $f(n)$
  - iv. move it to closed list
  - v. expand neighbors
  - vi. update costs
  - vii. repeat until goal found
- OPEN list: contains nodes yet to be explored.  
 implemented → priority queue, min heap

CLOSED list: contains already explored nodes.  
 prevents revisiting.

⑥ EXAMPLE:

| A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|
| 5 | 6 | 4 | 3 | 3 | 1 | 0 |

→ heuristic cost



## ⑦ Definition of Heuristic Function:

A heuristic is a rule or estimate used to guide search. It predicts closeness to goal.

## ⑧ types of Heuristic:

- i. Admissible Heuristic: never overestimates actual cost:
  - condition:  $h(n) \leq h^*(n)$
  - where:  $h^*(n)$  = true minimum cost
  - guarantees opt solution.
- ii. Non-Admissible Heuristic: may overestimate cost, faster sometimes, but optimality not guaranteed.
- iii. Consistent (Monotonic): satisfies triangle inequality:
  - $h(n) \leq c(n, n') + h(n')$
  - important for efficiency.



(9) Common Functions :

i. Manhattan Distance :

• used in grids.

$$|x_1 - x_2| + |y_1 - y_2|$$

• Movement : up, down, left, right .

ii. Euclidean Distance :

• straight line distance

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

iii. Diagonal distance :

• used when diagonal movement allowed .

(10) Completeness :

is A\* complete? Yes, if finite branching factor, positive edge costs .

(11) Optimality : Yes, if heuristic is admissible .

(12) Time complexity :  $O(b^d)$  b: branching factor  
d: depth  
can be exponentially .

(13) space complexity :  $O(b^d)$  memory usage  
drawback .

(14) Advantages : find optimal path, faster than uninformed search, uses heuristic intelligence used in gaming/navigation .

(15) Disadvantages : high memory consumption, performance depends on heuristic, can be slow huge spaces .

(16) Applications : i. GPS navigation - shortest path finding .

ii. Video Games : NPC finding .

iii. Robotics : robot motion planning .

iv. Maze Solving . v. Network Routing .

(17) Special Case :

• if  $h(n) = 0$ , uniform cost search (Dijkstra's)

• if  $g(n) = 0$ , greedy best first search



## BFS and DFS :

### ① BFS (Breadth First Search) :

graph/tree traversal algo explores nodes level by level.

### ② working (uses a queue FIFO) :

- start from source node
- mark visited
- Pushed into queue
- Remove front node
- visit all unvisited neighbors and push to queue.
- repeat until queue becomes empty.

### ③ Queue, visited array/set.

(level order traversal)

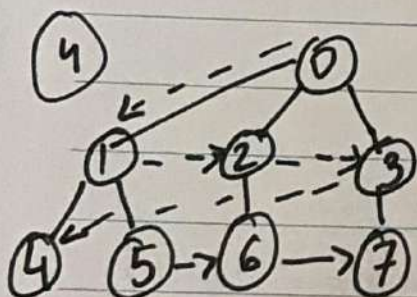
0 → 1 → 2 → 3 → 4 → 5 → 6 → 7

time complex :  $O(b^d)$

space " :  $O(b^d)$

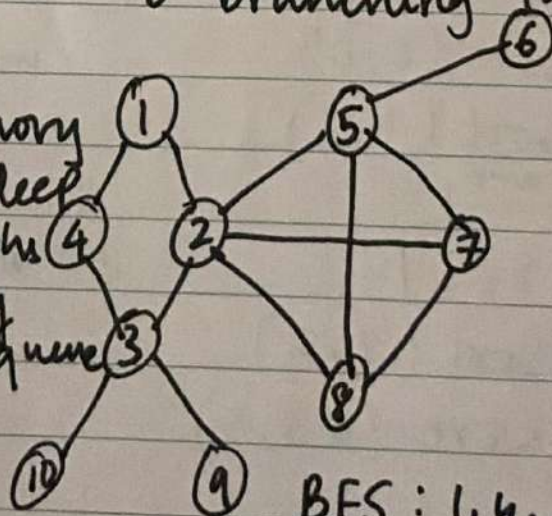
b : branching factor

d : depth of soln



### ④ disadv :

- high memory
- slow for deep graphs
- stores many nodes in queue



BFS : 1, 4, 2, 3, 5, 8, 7, 10, 9, 6

Queue :

|   |   |   |   |   |   |   |    |   |   |
|---|---|---|---|---|---|---|----|---|---|
| 1 | 4 | 2 | 3 | 5 | 8 | 7 | 10 | 9 | 6 |
|---|---|---|---|---|---|---|----|---|---|



Date:

`entry = tk.Entry (frame, font ("Arial", 12))`  
creates a single line input box where user can type their questions. It is placed inside the frame so it fits alongside the send button.

`entry.pack (side = tk.LEFT, fill = tk.X, expand = true, padx (0,8))`

Places the input box onto the window. `side = tk.LEFT` means place it on the left side of the frame. `fill = tk.X` & `expand` combine together to make it stretch horizontally space available. `padx (0,8)` 0 pixel on left & 8 on right

`entry.bind ("<Return>", send-message)`

connects the Enter i.e. `<Return>` key on the keyboard to the `send-message` function.

`bind()` links a keyboard or mouse event to a function.

`entry.focus()`

Automatically places the cursor in the input box once program opens.

`tk.Button (frame, text = "Send", font = ("Arial", 11), command = send-message).pack (*side = tk.RIGHT)`

`root.mainloop()`

It starts the event loop. A loop that keeps the window open & actively listening for any user interaction.



Date :

```
chat_box = tk.Text(root, font ("Arial", 11),  
wrap = tk.WORD, padx = 8, pady = 8, bg = #f5f5f5)
```

This creates a multi line text area where the conversation is displayed. `wrap = tk.WORD` means when the line is too long it wraps to the next line at the word boundary not mid word

```
chat_box.pack(padx = 10, fill = tk.BOTH,  
expand = True)
```

Places chat box onto the window. `padx = 10` adds a 10 pixel gap on the left & right side. `fill = tk.BOTH` means the widget stretches in BOTH horizontal & vertical direction. `expand = True` means it will grow to fill all remaining space in the window when resized.

```
chat_box.insert(tk.END, "Expert: Hello")
```

```
chat_box.config(state = tk.DISABLED)
```

```
frame = tk.Frame(root)
```

creates an invisible container that holds or groups multiple widgets together. Here it groups the input box & the send button.

```
frame.pack(padx = 10, pady = 8, fill = tk.X)
```

Places the frame onto the window. `fill = tk.X` means it stretches the frame across the full width horizontally.



Date :

chat\_box . yview (tk.END)

Automatically scrolls the chat box to the very end with latest message. yview controls the vertical scroll.

### GUI Window.

root = tk.TK()

This creates the main application window. tk.TK() is a constructor - it builds the base window object and stores it in a variable called root. Every other widget, button, labels, etc will be placed on this window.

root.title("Information Management Expert System")

Sets the text that appears in the title bar at the very top of the window.

root.geometry("500 x 500")

creates a window of the dimension 500 x 500 pixels.

wide      taller  
                  ↓            ↓

tk.Label(root, text = " ", font ("Arial", 13, "bold"), pady = 8).pack()

This line creates & places a Label onto the window. pady = 8 adds 8 pixels of spacing above & below the label. pack() is a function that places it immediately on the window.



Date :

### \*SEND Message

This function can be triggered by clicking send or by pressing enter.

```
def send_message (event = None):
```

```
    user_input = entry.get().strip() ← Removes any extra blank spaces
```

Reads anything that the user has written in the input box

```
    if not user_input:
```

```
        return
```

```
    chat_box.config (state = tk.NORMAL)
```

The NORMAL state allows the program to write into the chat box. If the chat\_box is disabled then the user cannot type into it directly. So this is done before every insert line.

```
    chat_box.insert (tk.END, "You: " + user_input + "\n")
```

Inserts the message into the chat\_box. tk.END means add the text at the very

END after everything already there.

```
    entry.delete (0, tk.END)
```

Deletes everything in the ~~cha~~ input\_box after the message has been sent.

tk.END means "clear upto the very end."

```
    solution = find_solution (user_input)
```

```
    chat_box.insert (tk.END, "Expert: " + solution + "\n\n")
```

```
    chat_box.config (state = tk.DISABLED)
```

This switches the chat\_box back into disabled state. This locks the chatbox so that the user cannot edit or delete the chat history.